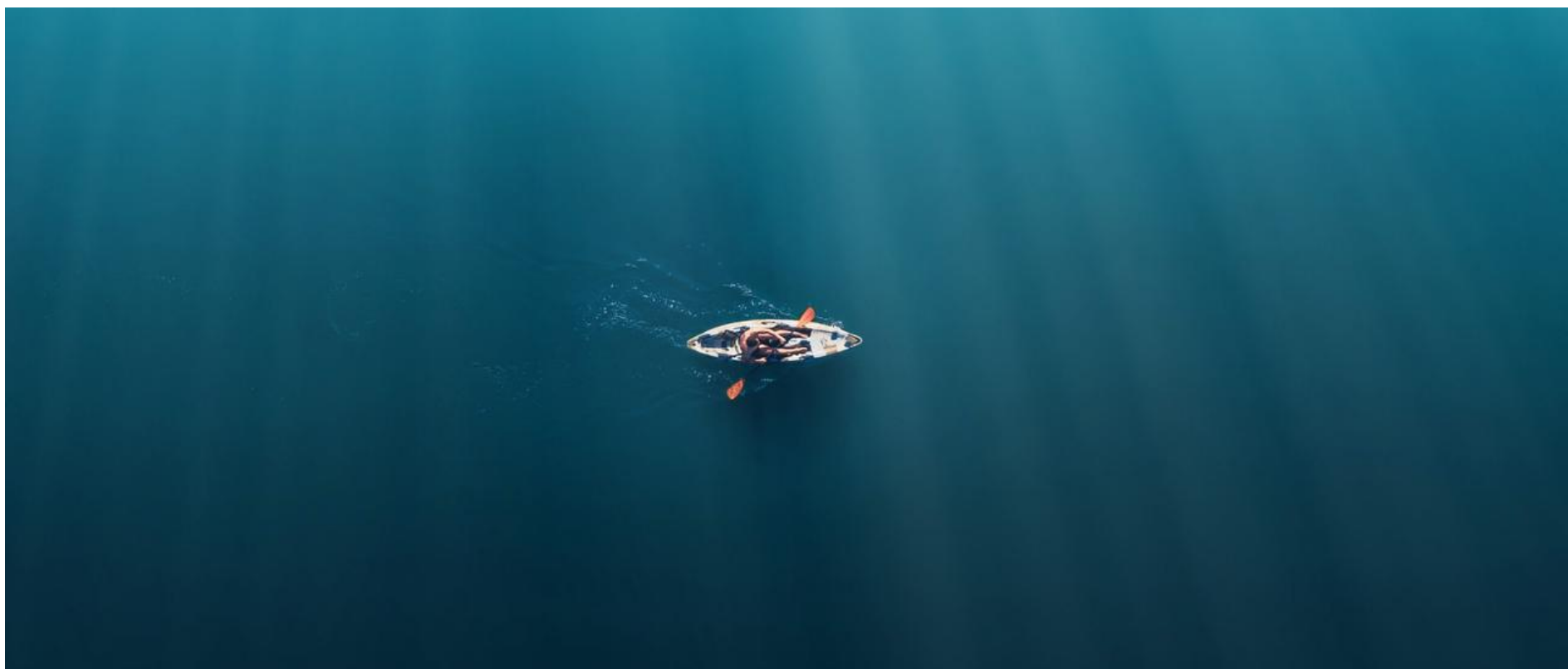




阿里&字节9月面试高频真题预测

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 25年下半年面试真题预测及解析](#)
- [📖 阿里/字节面试高频真题解析与9月考点预测](#)

2. 初中级面试必备

2.1 Symbol.Iterator

`Symbol.iterator` 为每一个对象定义了默认的迭代器。该迭代器可以被 `for...of` 循环使用。

Code block

```
1  const iterable1 = {};  
2  
3  iterable1[Symbol.iterator] = function* () {  
4    yield 1;  
5    yield 2;  
6    yield 3;  
7  };  
8  
9  console.log([...iterable1]);  
10 // Expected output: Array [1, 2, 3]
```

内置可迭代对象

Code block

```
1  const arr = [1, 2, 3];  
2  for (const item of arr) {  
3    console.log(item); // 1, 2, 3  
4  }  
5  
6  const str = "hello";  
7  for (const char of str) {  
8    console.log(char); // h, e, l, l, o  
9  }  
10  
11 const map = new Map([['a', 1], ['b', 2]]);  
12 for (const [key, value] of map) {  
13   console.log(key, value); // a 1, b 2  
14 }
```

使自定义对象可迭代

Code block

```
1  const myIterable = {  
2    data: [10, 20, 30],  
3    [Symbol.iterator]() {  
4      let index = 0;  
5      return {  
6        next: () => {  
7          if (index < this.data.length) {  
8            return { value: this.data[index++], done: false };  
9          }  
10         return { done: true };  
11       }  
12     };  
13   }  
14 };  
15  
16 for (const item of myIterable) {  
17   console.log(item); // 10, 20, 30  
18 }
```

1. 协议一致性：`Symbol.iterator` 方法必须返回一个迭代器对象，该对象包含一个 `next()` 方法。
2. `next()` 方法：每次调用 `next()` 返回一个包含两个属性的对象：

- `value`：当前迭代的值
 - `done`：布尔值，表示迭代是否完成
3. 自动调用：在以下情况下会自动调用 `Symbol.iterator` 方法：
- `for...of` 循环
 - 展开运算符 `...`
 - 解构赋值
 - `Array.from()`
 - `new Set()`，`new Map()` 等构造函数
4. 可提前终止：迭代器可以可选地实现 `return()` 方法，用于在迭代提前退出时执行清理工作

生成器函数简化迭代器

Code block

```
1  const myIterable = {
2    *[Symbol.iterator]() {
3      yield 1;
4      yield 2;
5      yield 3;
6    }
7  };
8
9  console.log([...myIterable]); // [1, 2, 3]
10
```

检查对象是否可迭代

Code block

```
1  function isIterable(obj) {
2    return obj != null && typeof obj[Symbol.iterator] === 'function';
3  }
4
5  console.log(isIterable([])); // true
6  console.log(isIterable({})); // false
```

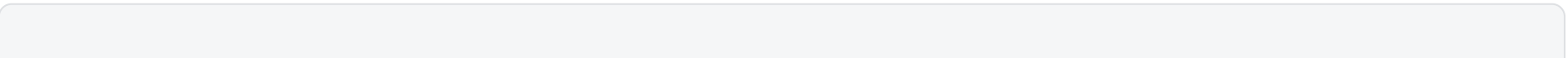
2.2 箭头函数

1. 没有自己的 `this`

箭头函数没有自己的 `this` 绑定，它会捕获其所在上下文的 `this` 值。

特性	传统函数	箭头函数
<code>this</code> 绑定时机	调用时确定	定义时确定
<code>this</code> 可变性	可改变	不可改变
适用场景	需要动态 <code>this</code> 的场景	需要固定 <code>this</code> 的场景

2. 不适合作为方法函数



```

1  const obj = {
2    name: "Alice",
3    traditionalFunc: function() {
4      console.log(this.name); // Alice
5    },
6    arrowFunc: () => {
7      console.log(this.name); // undefined (或 window.name 在浏览器中)
8    }
9  };
10
11 obj.traditionalFunc(); // 正确显示 "Alice"
12 obj.arrowFunc(); // 不显示或显示全局的 name

```

3. 继承外层作用域的 `this`

Code block

```

1  const obj = {
2    name: 'Alice',
3    sayName: function() {
4      // 传统函数, this 取决于调用方式
5      console.log(this.name);
6
7      setTimeout(function() {
8        console.log(this.name); // undefined (this 指向 window/global)
9      }, 100);
10
11     setTimeout(() => {
12       console.log(this.name); // 'Alice' (继承外层 this)
13     }, 100);
14   }
15 };
16
17 obj.sayName();

```

4. 无法通过 `call`/`apply`/`bind` 改变 `this`

Code block

```

1  const obj1 = { value: 1 };
2  const obj2 = { value: 2 };
3
4  const fn = () => console.log(this.value);
5
6  fn.call(obj1); // undefined (this 不会改变)
7  fn.apply(obj2); // undefined
8  const boundFn = fn.bind(obj1);
9  boundFn(); // undefined

```

5. 不能作为构造函数

Code block

```

1  const Person = (name) => {
2    this.name = name; // TypeError: 箭头函数不能用作构造函数
3  };
4
5  // 正确的方式是使用传统函数
6  function Person(name) {

```

```
7   this.name = name;
8 }
```

6. 没有 arguments 对象

箭头函数没有自己的 arguments 对象，但可以访问外围函数的 arguments。

Code block

```
1  function outer() {
2    const inner = () => {
3      console.log(arguments); // 访问 outer 的 arguments
4    };
5    inner();
6  }
7
8  outer(1, 2, 3); // 输出 [1, 2, 3]
```

7. 没有 prototype 属性

Code block

```
1  const arrow = () => {};
2  console.log(arrow.prototype); // undefined
```

8. 不能用作生成器函数

Code block

```
1  const gen = *() => {}; // SyntaxError
```

9. 适合回调函数

Code block

```
1  class Button {
2    constructor() {
3      this.text = 'Click me';
4      this.element = document.createElement('button');
5
6      // 传统函数需要 bind
7      this.element.addEventListener('click', function() {
8        console.log(this.text); // undefined (this 指向 DOM 元素)
9      });
10
11     // 箭头函数自动绑定
12     this.element.addEventListener('click', () => {
13       console.log(this.text); // "Click me"
14     });
15   }
16 }
```

- 在需要固定 this 的场景使用箭头函数（如回调函数）
- 需要动态 this 时使用传统函数
- 对象方法使用传统函数或方法简写语法
- 类方法使用传统函数或类字段箭头函数（需注意内存占用）

2.3 Cookie

2.3.1 Secure 和 HttpOnly 属性

Secure 属性和HttpOnly 属性

- Secure：表示只应通过被 HTTPS 协议加密过的请求发送给服务端；
- HttpOnly：JavaScript Document.cookie API 无法访问带有 HttpOnly 属性的 cookie；此类 Cookie 仅作用于服务器。例如，持久化服务器端会话的 Cookie 不需要对 JavaScript 可用，而应具有 HttpOnly 属性。

2.3.2 SameSite

SameSite 可以有下面三种值：

1. None：浏览器会在同站请求、跨站请求下继续发送 cookies，不区分大小写；
2. Strict：浏览器将只在访问相同站点时发送 cookie；
3. Lax：与 Strict 类似，但用户从外部站点导航至 URL 时（例如通过链接）除外。在新版本浏览器中，为默认选项，Same-site cookies 将会为一些跨站子请求保留，如图片加载或者 frames 的调用，但只有当用户从外部站点导航到 URL 时才会发送。如 link 链接；

大多数主流浏览器基本上已经将 SameSite 的默认值迁移至 Lax。如果想要指定 Cookies 在同站、跨站请求都被发送，现在需要明确指定 SameSite 为 None；

2.3.3 安全性

1. 使用 HttpOnly 属性可防止通过 JavaScript 访问 cookie 值；
2. 用于敏感信息（例如指示身份验证）的 Cookie 的生存期应较短，并且 SameSite 属性设置为Strict 或 Lax

2.4 柯里化

函数式编程：从完全调用 -> 不完全调用（闭包）
柯里化：函数元降元的方式

柯里化

Code block

```
1  const curry = (fn) => {
2    return function curried(...args) {
3      if (args.length >= fn.length) {
4        return fn.apply(this, args);
5      } else {
6        return (...args2) => curried.apply(this, args.concat(args2));
7      }
8    };
9  };
10 // 简单点的实现
11 const curry = fn =>
12   curried = (...args) =>
13     args.length >= fn.length
14     ? fn(...args): (...args2) => curried(...args, ...args2);
```

偏函数

Code block

```
1  const partial = (fn, ...presetArgs) =>
2    (...laterArgs) =>
3      fn(...presetArgs, ...laterArgs);
```

函数组合

Code block

```
1  const pipe = (...fns) =>
2    (initialValue) =>
3      fns.reduce((acc, fn) => fn(acc), initialValue);
4
5  // 使用示例
6  const add5 = x => x + 5;
7  const multiply3 = x => x * 3;
8  const square = x => x * x;
9
10 const transform2 = pipe(add5, multiply3, square);
11 console.log(transform2(2)); // ((2 + 5) * 3)^2 = 441
```

2.5 如何实现主题切换

Css var

- 纯CSS实现，性能好
- 支持实时切换无闪烁

Code block

```
1  :root {
2    --primary-color: #4285f4;
3    --secondary-color: #34a853;
4    --text-color: #202124;
5    --bg-color: #ffffff;
6  }
7
8  [data-theme="dark"] {
9    --primary-color: #8ab4f8;
10   --secondary-color: #81c995;
11   --text-color: #e8eaed;
12   --bg-color: #202124;
13 }
14
15 // js切换
16 function toggleTheme() {
17   const currentTheme =
18     html.getAttribute('data-theme');
19   html.setAttribute('data-theme', newTheme);
20   localStorage.setItem('theme', newTheme);
21 }
```

动态加载主题

- 主题完全隔离
- 可按需加载减少初始包体积

Code block

```
1  function loadTheme(themeName) {
2    const link = document.createElement('link');
3    link.rel = 'stylesheet';
4    link.href = `/themes/${themeName}.css`;
5    link.id = 'theme-style';
6  }
```

CSS 预处理器

- 编译时确定主题，性能最优
- 适合主题数量固定的场景

Code block

```
1  $themes: (
2    light: (
3      primary: #4285f4,
4      bg: #ffffff,
5      text: #202124
6    ),
7    dark: (
8      primary: #8ab4f8,
9      bg: #202124,
10     text: #e8eaed
11   )
12 );
13
14 @mixin theme($property, $key) {
15   @each $theme-name, $theme-map in $themes {
16     [data-theme="#{$theme-name}"] & {
17       #{$property}: map-get($theme-map, $key);
18     }
19   }
20 }
```

CSS-in-JS、tailwind

Code block

```
1  import { ThemeProvider } from 'styled-
2    components';
3
4  const lightTheme = {
5    colors: {
6      primary: '#4285f4',
7      bg: '#ffffff',
8      text: '#202124'
9    }
10 };
```



```
7     const oldLink =
      document.getElementById('theme-style');
8     if (oldLink) {
9       document.head.replaceChild(link, oldLink);
10    } else {
11      document.head.appendChild(link);
12    }
13  }
```

```
10
11  const darkTheme = {
12    colors: {
13      primary: '#8ab4f8',
14      bg: '#202124',
15      text: '#e8eae6'
16    }
17  };
```

3. 高阶面试必备

3.1 性能优化

定量指标：

1. Largest Contentful Paint : 最大内容绘制，测量加载性能。理想值 < 2.5 秒。
2. First Input Delay : 首次输入延迟，测量交互性。理想值 < 100 毫秒。
3. Cumulative Layout Shift : 累积布局偏移，测量视觉稳定性。理想值 < 0.1。

3.1.1 编译构建时优化

1. **体积优化**：terser 压缩、混淆
2. **资源优化**
 - 图片：webp等、srcset 基于dpi尺寸
 - 字体：预加载关键字体: `<link rel="preload" as="font">`。
 - 懒加载：script defer。
 - CDN
3. **减少 HTTP 请求**
 - **文件合并**: 在 HTTP/1.1 时代常用，但在 HTTP/2+ 下需权衡（多路复用减少了合并的必要性，合并可能影响缓存粒度）。小图标合并成雪碧图仍有价值。
 - **内联关键 CSS/JS**: 首屏内容 & 骨架屏 所必需的小段 CSS/JS 直接内联在 HTML 中，避免阻塞渲染的请求。
4. **利用浏览器缓存**
 - **强缓存 (Cache-Control, Expires)**: 为静态资源设置较长的过期时间 (e.g., `max-age=31536000`, `immutable`)。
 - **协商缓存 (ETag, Last-Modified)**: 确保用户代理能有效验证资源是否过期。
 - **Service Worker**: 实现更精细的缓存策略（网络优先、缓存优先等），支持离线访问。PWA 的核心。
5. **减少资源体积**
 - Code Splitting
 - Tree Shaking
 - Webpack Bundle Analyzer。
6. **预加载、预连接、预获取**

3.1.2 运行时优化

1. **优化 CSS**
 - a. 避免深层嵌套选择器: 降低匹配复杂度。
 - b. 减少过度使用昂贵的属性: 如 `box-shadow`, `border-radius`, `filter`
2. **优化 JavaScript 执行**
 - a. 避免长任务: 将耗时 JS 任务分解成小块 (使用 `setTimeout`, `requestAnimationFrame`, `requestIdleCallback`, Web Workers)。
 - b. 防抖和节流: 控制高频事件 (resize, scroll, input) 的处理频率。
 - c. Web Workers: 将复杂的计算任务移出主线程，避免阻塞 UI 渲染和交互响应。
 - d. 优化事件处理: 使用事件委托，及时移除不再需要的事件监听器。
 - e. 虚拟列表: 对于超长列表，仅渲染可视区域内的项 (`react-window`, `vue-virtual-scroller`)。
3. **优化渲染流程**:
 - a. 减少重排和重绘: 集中修改 DOM 样式 (使用 `class` 切换)，避免逐个修改。离线操作 DOM (使用 `DocumentFragment`, `cloneNode`, 隐藏元素修改完再显示)。
 - b. GPU 加速: 对动画元素使用 `transform` 和 `opacity` 属性，它们通常在合成层处理，避免触发重排重绘。
4. **框架优化: 避免不必要的渲染**
 - a. React: 使用 `React.memo`, `PureComponent`, `useMemo`, `useCallback`。优化 Context 的使用。
 - b. Vue: 利用 `v-once`, `v-memo` (Vue 3), 计算属性缓存，合理设计组件。

- `<link rel="preload">` : ** 强制浏览器提前加载关键资源 (字体、首屏图片、关键 JS/CSS)。
- `<link rel="preconnect">` : ** 提前建立与关键第三方源 (CDN, API 域名, 分析服务) 的连接 (DNS 查找、TCP 握手、TLS 协商)。
- `<link rel="prefetch">` : ** 在浏览器空闲时预取用户下一步可能需要的资源 (如下一页的内容、非首屏图片)。
- `<link rel="dns-prefetch">` : ** 提前解析域名 DNS (比 `preconnect` 更轻量)。

7. HTTP/2 或 HTTP/3

- 确保服务器和 CDN 支持 HTTP/2 (多路复用、头部压缩、服务器推送 - 需谨慎使用) 或更先进的 HTTP/3 (基于 QUIC, 改进弱网性能)。

3.2 Webpack hash 的作用对比

- fullhash: 所有构建产物均使用相同的 hash。这意味着修改任何一个文件, 所有文件的 hash 值都将会被改变
- chunkhash: 同一 chunk 下的文件使用相同的 hash。修改在同一个 chunk 中的文件, 同属该 chunk 的所有文件 hash 值都会改变
- contenthash: 不同的文件使用不同的 hash。每个文件都会根据自己的文件内容生成 hash 值, 互不干扰

4. 9月面试如何准备?

3.1.3 用户体验

1. 骨架屏
2. 占位符
3. 进度条
4. 交互优化 Hover click etc
5. 资源缓存 LRU



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



5. 算法真题解析

5.1 JSON转树 树转JSON

Json 转树

Code block

```
1 function convertToTree(data) {
2     // 创建节点映射表 (id -> 节点)
3     const nodeMap = new Map();
4
5     // 第一次遍历: 创建所有节点的映射并初始化
    children数组
6     data.forEach(item => {
7         nodeMap.set(item.id, { ...item,
            children: [] });
8     });
9
10    const roots = []; // 存储根节点
11
12    // 第二次遍历: 构建树结构
13    data.forEach(item => {
14        const node = nodeMap.get(item.id);
15        if (item.parentId === null) {
16            roots.push(node); // 根节点直接加入
            结果
17        } else {
18            // 找到父节点并将当前节点加入其
            children
19            const parent =
                nodeMap.get(item.parentId);
20            if (parent) {
21                parent.children.push(node);
22            }
23        }
24    });
25
26    return roots;
27 }
28 时间复杂度 O(n):
29 两次独立的数组遍历 (O(n) + O(n) = O(n))
30 Map 的插入和查找操作均为 O(1)
31 空间复杂度 O(n):
32 使用 Map 存储所有节点 (空间 O(n))
33 结果树复用节点对象
```

树转json

Code block

```
1 function convertToJson(treeData) {
2     const result = [];
3
4     function traverse(node) {
5         // 创建当前节点的副本, 不包含children属性
6         const { children, ...rest } = node;
7         result.push(rest);
8
9         // 递归处理子节点
10        if (children && children.length > 0) {
11            children.forEach(child =>
                traverse(child));
12        }
13    }
14
15    // 遍历每棵树的根节点
16    treeData.forEach(root => traverse(root));
17
18    return result;
19 }
```

DFS 深度优先

- 1. 时间复杂度: O(n), 其中n是树中所有节点的总数, 每个节点只被访问一次
- 2. 空间复杂度: O(h), 其中h是树的最大深度, 这是递归调用的栈空间

5.2 无序递增数组

生成一个长度为 m 的数组, 数组元素是随机生成的、不重复的、递增的整数, 最大值不超过 n

时间复杂度: O(m)

- a. Set.add 复杂度O(1)
- b. 循环次数O(m)

空间复杂度: O(mlogm)

- a. Array.from 复杂度 O(m)
- b. Sort 快排 O(mlogm)

```

1  function generateRandomArray(m, n) {
2      if (m >= n) {
3          throw new Error("m must be less than n");
4      }
5
6      // 使用Set确保元素不重复
7      const set = new Set();
8      while (set.size < m) {
9          // 生成[0, n]范围内的随机整数
10         const num = Math.floor(Math.random() * (n + 1));
11         set.add(num);
12     }
13
14     // 将Set转为数组并排序（升序）
15     const array = Array.from(set).sort((a, b) => a - b);
16     return array;
17 }
18
19 // 示例用法
20 const m = 5;    // 数组长度
21 const n = 10;   // 最大值（包含n）
22 const randomArray = generateRandomArray(m, n);
23 console.log(randomArray); // 输出示例: [0, 3, 5, 7, 9]

```

5.3 前序中序后序排列

1. 时间复杂度：所有实现均为 $O(n)$ ，每个节点只被访问一次
2. 空间复杂度：
 - 递归实现： $O(h)$ ， h 为树的高度（递归调用栈）
 - 迭代实现： $O(n)$ ，最坏情况下需要存储所有节点

前序

```

Code block
1  // 递归
2  function
3  preOrderTraversal(root) {
4      const result = [];
5
6      function traverse(node)
7      {
8          if (!node) return;
9
10         result.push(node.value);
11         // 访问根节点
12         traverse(node.left);
13         // 遍历左子树
14         traverse(node.right);
15         // 遍历右子树
16     }
17
18     traverse(root);
19     return result;
20 }
21
22 // 使用栈
23 function
24 preOrderTraversal(root) {

```

中序

```

Code block
1  // 递归
2  function
3  inOrderTraversal(root) {
4      const result = [];
5
6      function traverse(node)
7      {
8          if (!node) return;
9
10         traverse(node.left);
11         // 遍历左子树
12         result.push(node.value);
13         // 访问根节点
14         traverse(node.right);
15         // 遍历右子树
16     }
17
18     traverse(root);
19     return result;
20 }
21
22 // 使用栈
23 function
24 inOrderTraversal(root) {

```

后序

```

Code block
1  // 递归
2  function
3  postOrderTraversal(root) {
4      const result = [];
5
6      function traverse(node)
7      {
8          if (!node) return;
9
10         traverse(node.left);
11         // 遍历左子树
12         traverse(node.right);
13         // 遍历右子树
14         result.push(node.value);
15         // 访问根节点
16     }
17
18     traverse(root);
19     return result;
20 }
21
22 // 使用栈
23 function
24 postOrderTraversal(root) {

```

```

18   if (!root) return [];
19
20   const result = [];
21   const stack = [root];
22
23   while (stack.length) {
24     const node =
stack.pop();
25     result.push(node.value);
26
27     // 先压入右节点，再压入左
    节点
28     if (node.right)
stack.push(node.right);
29     if (node.left)
stack.push(node.left);
30   }
31
32   return result;
33 }

```

```

18   const result = [];
19   const stack = [];
20   let current = root;
21
22   while (current ||
stack.length) {
23     // 先遍历到最左节点
24     while (current) {
25       stack.push(current);
26       current =
current.left;
27     }
28
29     current = stack.pop();
30
31     result.push(current.value)
;
32     current =
current.right;
33
34   return result;
35 }

```

```

18   if (!root) return [];
19
20   const result = [];
21   const stack = [root];
22   const visited = new
Set();
23
24   while (stack.length) {
25     const node =
stack[stack.length - 1];
26
27     // 如果左右子节点都已访问
    过，则可以访问当前节点
28     if (
29       (node.left === null
&& node.right === null) ||
30       (visited.has(node.left)
&& visited.has(node.right)
31       ) {
32
33       result.push(node.value);
34       visited.add(node);
35       stack.pop();
36     } else {
37       // 先压入右节点，再压
    入左节点
38       if (node.right &&
!visited.has(node.right))
stack.push(node.right);
39       if (node.left &&
!visited.has(node.left))
stack.push(node.left);
40     }
41
42   return result;
43 }

```

5.4 红绿灯

Code block

```

1   function red() {
2     console.log('红灯亮');
3   }
4   function yellow() {
5     console.log('黄灯亮');
6   }
7   function green() {
8     console.log('绿灯亮');
9   }
10

```

Code block

```

1   function light(callback, timer) {
2     return new Promise(resolve => {
3       setTimeout(() => {
4         callback?.;()

```

Code block

```

1   async function lightStep() {
2     await light(red, 3000);
3     await light(yellow, 2000);
4     await light(green, 1000);

```

```

5         resolve();
6     }, timer);
7 })
8 }
9
10 function lightStep() {
11     Promise.resolve().then(() => {
12         return light(red, 3000);
13     }).then(() => {
14         return light(yellow, 2000);
15     }).then(() => {
16         return light(green, 1000);
17     }).finally(() => {
18         return lightStep();
19     });
20 }

```

```

5     await lightStep();
6 }
7

```

5.5 实现有并行限制的 Promise 调度器

Code block

```

1  class Scheduler {
2      constructor(max) {
3          // 最大可并发任务数
4          this.max = max;
5          // 当前并发任务数
6          this.count = 0;
7          // 阻塞的任务队列
8          this.queue = [];
9      }
10
11     async add(fn) {
12         if (this.count >= this.max) {
13             // 若当前正在执行的任务，达到最大容量max
14             // 阻塞在此处，等待前面的任务执行完毕后将
15             // resolve弹出并执行
16             await new Promise(resolve =>
17                 this.queue.push(resolve));
18             // 当前并发任务数++
19             this.count++;
20             // 使用await执行此函数
21             const res = await fn();
22             // 执行完毕，当前并发任务数--
23             this.count--;
24             // 若队列中有值，将其resolve弹出，并执行
25             // 以便阻塞的任务，可以正常执行
26             this.queue.length && this.queue.shift();
27             // 返回函数执行的结果
28             return res;
29         }
30     }
31 }

```

Code block

```

1  // 延迟函数
2  const sleep = time => new Promise(resolve =>
3      setTimeout(resolve, time));
4
5  // 同时进行的任务最多2个
6  const scheduler = new Scheduler(2);
7
8  // 添加异步任务
9  // time: 任务执行的时间
10 // val: 参数
11 const addTask = (time, val) => {
12     scheduler.add(() => {
13         return sleep(time).then(() =>
14             console.log(val));
15     });
16 }
17
18 addTask(1000, '1');
19 addTask(500, '2');
20 addTask(300, '3');
21 addTask(400, '4');
22
23 // 2
24 // 3
25 // 1
26 // 4

```

5.6 模拟大数相加

Code block

```

1  var addStrings = function (num1, num2) {
2      let l1 = num1.length - 1

```

```
3   let l2 = num2.length - 1
4   let add = 0
5   let total = ''
6   while (l1 >= 0 || l2 >= 0 || add) {
7     let t = (num1[l1] * 1 || 0) + (num2[l2] * 1 || 0) + add
8     total = t % 10 + total
9     add = t > 9 ? 1 : 0
10    l1--
11    l2--
12  }
13  return total
14  };
```